

Fooling Machine's Eyes: Unicode Modifier Letter Evasion Attack

Artifact Reproduction and Security Analysis Report

Submitted By

Name: *Shalvin Didwaniya*

Student ID : *2023UAI1811*

Course: *Information Security*

Abstract

This report presents the reproduction and analysis of the research paper *"Fooling Machine's Eyes: Unicode Modifier Letter Evasion Attack"* by Chao Gao, Guanglu Sun, Xin Liu, and Feiyan Liu from Harbin University of Science and Technology, China. The paper investigates how Unicode modifier letters can be abused to evade machine-based detection systems while remaining visually understandable to humans.

The project focuses on reproducing the official artifact provided by the authors, executing the provided binaries, generating the expected experimental outputs, and validating the reproduced results against the expected reports. The reproduction process included execution of the Unicode analysis tool (`analyzer.exe`) and the Unicode Modifier Converter (`checker.exe`). The generated report was verified against the expected artifact output using SHA256 hash comparison. The project successfully reproduced the character scanning experiment from the paper and validated the Unicode equivalence behavior discussed by the authors.

1. Introduction

Modern security systems heavily depend on string comparison mechanisms for malware detection, phishing detection, URL validation, and threat intelligence systems. However, Unicode introduces thousands of visually similar characters that may not be processed consistently by machine-based systems.

The research paper *"Fooling Machine's Eyes: Unicode Modifier Letter Evasion Attack"* studies how Unicode modifier letters can be used to bypass detection systems while remaining visually understandable to humans.

For example:

Normal string:

```
password
```

Unicode modifier version:

```
password
```

Although humans can still interpret the modified text correctly, some systems may fail to detect or compare the string properly. This creates opportunities for:

- Malware obfuscation
- Signature evasion
- Phishing attacks
- Identifier spoofing
- Security filter bypasses

The paper investigates how Windows APIs such as `CompareStringW()` and `IdnToAscii()` handle Unicode modifier characters.

2. Objectives

The primary objectives of this project were:

1. To understand Unicode modifier letter evasion attacks.
 2. To reproduce the official artifact provided by the authors.
 3. To execute the Unicode character scanning experiment.
 4. To generate the expected report output.
 5. To validate reproduced outputs against expected outputs.
 6. To understand the security implications of Unicode equivalence vulnerabilities.
-

3. Background

3.1 Unicode Modifier Letters

Unicode modifier letters are special Unicode characters that visually resemble normal Latin letters. These characters are visually similar to normal characters but internally represented differently.

Examples:

ASCII Character	Unicode Modifier
a	ᵃ
e	ᵉ
t	ᵗ
i	ᵢ
n	ⁿ

3.2 CompareStringW()

`CompareStringW()` is a Windows API function used for:

- String comparison
- Equality checking
- Sorting operations
- Case-insensitive comparison

Many applications internally rely on this API.

The paper demonstrates that under certain comparison settings, Unicode modifier letters may be treated as equivalent to their ASCII counterparts.

Example:

```
CompareStringW(ˆ, ˆ)
```

may return equivalence under specific flags.

3.3 Security Implications

This behavior can create several security vulnerabilities:

Malware Evasion

A malicious file may rename itself using modifier letters:

```
cmd.exe → CMD.exe
```

Humans perceive both similarly, but signature-based systems may fail to detect the modified variant.

Phishing and Spoofing : Unicode-based lookalike domains may evade validation systems.

Detection Bypass : Security tools that rely on exact string matching may miss obfuscated variants.

4. Paper Overview

The paper introduces a vulnerability analysis framework and Unicode normalization tool to systematically analyze modifier letter behavior in Windows systems.

The provided artifact contains:

Component	Purpose
analyzer.exe	Vulnerability analysis tool
checker.exe	Unicode Modifier Converter tool
Reports	Previously generated experimental reports
Claims	Reproducibility claim scripts and expected outputs

The analyzer supports multiple modes:

Mode	Description
1	Validate CompareStringW behavior
2	Validate IdnToAscii behavior
3	Parameter space scanning
4	Character equivalence scanning
51/52	Program scanning

The reproduction focused primarily on:

- Mode 4: Character equivalence scanning
- UMC Tool verification using `checker.exe`

5. Experimental Setup

5.1 System Configuration

Parameter	Value
Operating System	Windows 10/11
Environment	PowerShell + VS Code
Artifact Source	Official Zenodo Artifact
Architecture	x64

5.2 Artifact Structure

The artifact contained:

```
artifact/  
├─ Release/  
│   ├── analyzer.exe  
│   ├── checker.exe  
│   └─ libumlec.dll  
├─ Report/  
├─ Case/  
├─ claims/  
└─ umla.sln
```

The Release directory contained the compiled binaries required for reproduction.

6. Methodology

6.1 Running the Analyzer Tool

The analyzer was executed from:

```
artifact/Release/
```

Execution command:

```
.\analyzer.exe
```

The following menu appeared:

```
Unicode Modifier Letter Evasion Attack Analysis Tool  
  
Please choose an item:  
1. Show validation result of CompareStringW() with modifier letter arguments.  
2. Show validation result of IdnToAscii() with modifier letter arguments.  
3. Run CompareStringW parameter space scanning.  
4. Run CompareStringW character scanning(single character equal-comparison).
```

6.2 Character Scanning Experiment

The reproduction focused on:

```
Option 4 - Character Scan
```

This experiment systematically scans Unicode modifier characters and compares them with ASCII characters using `CompareStringW()`.

The goal is to discover equivalence mappings between Unicode modifier letters and standard ASCII characters.

6.3 Generated Output

After execution, the analyzer generated:

```
TestReport2.txt
```

inside:

```
artifact/Report/
```

This report contains discovered equivalence mappings and scan results.

7. Unicode Modifier Converter (UMC Tool)

The `checker.exe` tool demonstrates Unicode modifier normalization.

Execution command:

```
.\checker.exe
```

Example input:

```
t e s t
```

Expected conversion:

```
test
```

This validates that modifier letters can be normalized back to ASCII representations.

8. Reproduction Verification

8.1 Expected Output Comparison

The expected report provided by the authors was located at:

```
claims/claim2_character_scan/expected/TestReport2.txt
```

The generated report was compared against the expected report.

8.2 SHA256 Hash Verification

SHA256 hashes were computed for:

- Generated TestReport2.txt
- Expected TestReport2.txt Both hashes matched exactly.

This confirms: **Exact Reproduction of Experimental Output**

8.3 Verification Summary

Verification Item	Result
analyzer.exe execution	PASS
Character scan completed	PASS
TestReport2 generated	PASS
checker.exe executed	PASS
Unicode equivalence confirmed	PASS
SHA256 comparison matched	PASS

9. Additional Evaluation (Extension Work)

As an additional experiment, several custom Unicode modifier attack strings were evaluated.

Examples:

Original	Unicode Modifier Variant
password	p ^a ssword
admin	adm'n
malware	m ^a lw ^a re
trojan	tr ^o j ^a n

The goal was to observe whether visually readable Unicode variants could create ambiguity or bypass detection logic.

This additional evaluation demonstrates the practical applicability of the paper's findings.

10. Discussion

The experiments demonstrate that Unicode modifier letters create inconsistencies between human interpretation and machine processing.

The research highlights an important class of security risks:

- Security systems relying on exact string comparison may become vulnerable.
- Unicode normalization inconsistencies can enable evasion attacks.
- Detection systems must properly normalize Unicode before comparison.

The successful reproduction confirms that the vulnerability is reproducible using the official artifact provided by the authors.

11. Challenges Faced

Several practical issues were encountered during reproduction:

- Initial repository lacked compiled binaries.
- PowerShell command behavior differed from CMD behavior.
- Artifact structure required exploration before execution.
- Large output files required hash-based comparison instead of manual inspection.

These challenges were resolved successfully during the reproduction process.

12. Conclusion

This project successfully reproduced the Unicode character equivalence scanning experiment presented in the paper *"Fooling Machine's Eyes: Unicode Modifier Letter Evasion Attack."*

Using the official Zenodo artifact provided by the authors, the analyzer tool was executed, the character scanning experiment was reproduced, and the generated report matched the expected report exactly using SHA256 hash verification. The project demonstrates how Unicode modifier letters can create security vulnerabilities in systems relying on string comparison APIs such as `CompareStringW()` .

References

1. Chao Gao, Guanglu Sun, Xin Liu, and Feiyan Liu, "Fooling Machine's Eyes: Unicode Modifier Letter Evasion Attack," Annual Computer Security Applications Conference (ACSAC), 2025.
 2. Official Zenodo Artifact provided by the authors.
-